

Chapter 16: Enum & Sealed Classes

By Ellen Shapiro

Sometimes you'll have a piece of information that could (or at least, should) have only one of a limited number of potential values. Using what you already know, you could make a `List` of all the acceptable values for that piece of information, and walk (or *enumerate*) through each value, one-by-one, to see if your new piece of information matches one of the expected values.

If you think that sounds boring and repetitive, you're not alone. This is why the concept of the **enum** was invented.

Note: There is some debate over how to pronounce the word **enum**. Since it derives from “enumeration,” some people pronounce it *ee-noom*. Some people pronounce it *ee-numb*, since in its shortened form, it looks a lot more like the prefix to the word “number.”

This book takes no position on which of these is the preferred pronunciation, but you should note that both pronunciations are used commonly, and people tend to feel quite strongly about which pronunciation is the “correct” one. Caveat coder.

In Kotlin, as in many other programming languages, an enum is its own specialized type, indicating that something has a number of possible values.

One big difference in Kotlin is that enums are made by creating an **enum class**. You get a number of interesting pieces of functionality that enums in other languages don't necessarily have. As you work through this chapter, you'll learn about some of the most commonly-used bits of functionality and how to take advantage of them as you work in Kotlin.

To get started, open the starter project for this chapter and dig in.

Creating your first enum class

Open up **main.kt**. Above the `main()` function, define a new enum class:

```
enum class DayOfTheWeek {  
    // more code goes here  
}
```

Next, replace the comment by adding a comma-separated list of **cases**, or individual values, for the day of the week:

```
Sunday,  
Monday,  
Tuesday,  
Wednesday,  
Thursday,  
Friday,  
Saturday
```

In the `main()` function, replace the existing `println()` statement with one which goes through and prints out some information you get for free with any enum class:

```
for (day in DayOfTheWeek.values()) {  
    println("Day ${day.ordinal}: ${day.name}")  
}
```

Run the updated `main()` function, and it should print out the following:

```
Day 0: Sunday  
Day 1: Monday  
Day 2: Tuesday  
Day 3: Wednesday  
Day 4: Thursday  
Day 5: Friday
```

Day 6: Saturday

Neat! So what did you just get for free from Kotlin by declaring `DaysOfTheWeek` to be an enum class?

- The `values()` companion function on the enum class gives you a `List` of all the declared cases in the class, making it easy to go through all possibilities, and also to find out how many possibilities exist.
- The `ordinal` property of each case gives that case's index in the list of declared cases. You'll note from what's printed out that the order is zero-indexed.
- The `name` property of each case takes the name of the case in code and gives back the `String` value of that name.

A lot of this behavior is possible because enum classes are, well, classes. Each case is an instance of the class, so things like compiler-generated companion object functions for the class itself and individual properties for each instance are possible. Additionally, because these properties return objects of their own, you can use other functionality like getting the day based on a passed-in integer index.

For example, let's say your colleagues working somewhere else in the code tell you that they'll hand you an integer representing the day of the week. You could use the functionality of `List`, with which you're already familiar, to get the value at the appropriate index.

Add the following to the `main()` function to see this in action:

```
val dayIndex = 0
val dayAtIndex = DaysOfTheWeek.values()[dayIndex]
println("Day at $dayIndex is $dayAtIndex")
```

Run the `main()` function again, and at the end, you'll see:

```
Day at 0 is Sunday
```

If you want, you can even change the index of the day to update the value returned. Make sure not to go beyond the length of `values()`, as that will throw an `ArrayIndexOutOfBoundsException`, just like it will with any other list in Kotlin.

Another nice piece of functionality you get for free is the `valueOf()` method, which takes a `String` and returns the enum instance matching that string.

Add the following to the bottom of the `main()` function:

```
val tuesday = DayOfTheWeek.valueOf("Tuesday")
println("Tuesday is day ${tuesday.ordinal}")
```

Run the `main()` function, and at the end of the output you'll see:

```
Tuesday is day 2
```

Neat! Now, the eagle-eyed among you may have noticed that the `valueOf()` function doesn't return a nullable. So what happens when you try to get the value of an enum case that doesn't exist? Let's find out.

Add the following lines to the `main()` function:

```
val notADay = DayOfTheWeek.valueOf("Blernsday")
println("Not a day: $notADay")
```

Run `main()` again, and:

```
Exception in thread "main" java.lang.IllegalArgumentException:
No enum constant DayOfTheWeek.Blernsday
    at java.lang.Enum.valueOf(Enum.java:238)
    at DayOfTheWeek.valueOf(main.kt)
    at MainKt.main(main.kt:23)
```

Nooooo! Weren't Kotlin's nullables supposed to save us from these "thing doesn't exist" exceptions!?

The designers of Kotlin decided that trying to access an enum case which doesn't exist, akin to accessing an index outside the bounds of an array, was enough of an error that an exception should be thrown. So that stopped your process dead in its tracks.

Delete the last two lines you added looking for "Blernsday" so the rest of your code runs.

Updating case order

Another nice thing about enum classes is that if you find out something needs to be in a different order from a zero-indexed perspective, it's easy to make that change.

For instance, a week is defined from Sunday until Saturday in the United States, as the `DaysOfTheWeek` enum does currently. However, in Europe, weeks generally go from Monday until Sunday. Standards — they're great, eh?

Imagine again that you'll be receiving information about the day of the week from somewhere as an integer value. But this time, instead of receiving it from American colleagues, you'll be receiving it from some European colleagues based on their own understanding of day of the week indexing.

The nice thing about using an enum class is that making this adjustment is super-easy and only requires you to change the order of the list of cases.

In the list of cases for **DaysOfTheWeek**, move Sunday down to the bottom of the list:

```
Monday,  
Tuesday,  
Wednesday,  
Thursday,  
Friday,  
Saturday,  
Sunday;
```

Make sure to add a comma after Saturday, and replace the comma after Sunday with a semicolon. Lists of enum cases are comma-separated, so anything that isn't the very last case has to have a comma. However, you don't leave a comma on the very last case, or the compiler won't understand that you've reached the end of the list.

If you're just making a simple list of cases, the semicolon at the end isn't necessary. However, since you're planning to add more functionality to this enum class, you need to add the semicolon to ensure that the compiler sees the list of cases has ended, and that other functionality declarations have begun.

Now, run the `main()` function again, and the output will update:

```
Day 0: Monday  
Day 1: Tuesday  
Day 2: Wednesday  
Day 3: Thursday  
Day 4: Friday  
Day 5: Saturday  
Day 6: Sunday  
Day at 0 is Monday  
Tuesday is day 1
```

Without changing any of the code in `main()`, the underlying values for each day's `ordinal` property have been updated to reflect the new order of the cases. Sweet!

Enum class properties and functions

Like other classes, enum classes can have properties and functions. You can even set them up to be passed in as part of the constructor for each case.

As an example, let's make it simple and easy to tell if a given day is on the weekend. Add a Boolean property to the constructor:

```
enum class DayOfTheWeek(val isWeekend: Boolean)
```

Since you've added this property to the constructor without assigning it a default value, you'll need to pass in a value for the `isWeekend` property for each case you're creating. Update your list of cases to use the constructor to set the `isWeekend` value for each case:

```
Monday(false),  
Tuesday(false),  
Wednesday(false),  
Thursday(false),  
Friday(false),  
Saturday(true),  
Sunday(true);
```

Next, update the first print statement in the `main()` function so it also prints out whether the day being logged is a weekend day or not:

```
println(  
    "Day ${day.ordinal}: ${day.name}, is weekend: $  
    {day.isWeekend}"  
)
```

Run the `main()` function again, and you'll see the results based on the values you passed in with the constructor:

```
Day 0: Monday, is weekend: false  
Day 1: Tuesday, is weekend: false  
Day 2: Wednesday, is weekend: false  
Day 3: Thursday, is weekend: false  
Day 4: Friday, is weekend: false  
Day 5: Saturday, is weekend: true  
Day 6: Sunday, is weekend: true
```

You can also use default values in constructors the same way you can with other classes. Update the constructor so that the default value of `isWeekend` is `false`:

```
enum class DayOfTheWeek(val isWeekend: Boolean = false)
```

Now, you can delete the (false) off of all the non-weekend days, since that's the default value of the isWeekend parameter:

```
Monday,  
Tuesday,  
Wednesday,  
Thursday,  
Friday,  
Saturday(true),  
Sunday(true);
```

Run the main() function again, and you'll get exactly the same output as before, but now your list of cases in code is slightly easier to read.

Like other classes in Kotlin, enum classes can have companion objects to do things that don't depend on a specific instance of the class. For example, let's say you want to find out which day today is in your DayOfTheWeek enum.

You can add a companion object with a function which calculates that, just as you can do with any other class.

In the DayOfTheWeek enum class, add a companion object and a skeleton of the function you're about to add:

```
companion object {  
    fun today(): DayOfTheWeek {  
        // Code goes here  
    }  
}
```

Here, you'll want to take advantage of Kotlin's interoperability with Java to use the battle-tested Java Calendar class to access information about the current day, then calculate how that translates to your enum class:

```
// 1  
val calendarDayOfWeek =  
    Calendar.getInstance().get(Calendar.DAY_OF_WEEK)  
// 2  
var adjustedDay = calendarDayOfWeek - 2  
// 3  
val days = DayOfTheWeek.values()  
if (adjustedDay < 0) {  
    adjustedDay += days.count()  
}  
// 4  
val today = days.first { it.ordinal == adjustedDay }  
return today
```

Note: You need to add the import, `import java.util.*`, to the top of your file to use `Calendar`.

What's happening in this code?

1. You're using the Java `Calendar`'s shared instance, you get the current day of the week, according to the `Calendar` class.
2. Because the Java `Calendar` class thinks (a) weeks start on Sunday and (b) weeks are 1-indexed instead of 0-indexed, you adjust the index returned by subtracting 2: 1 to account for the indexing change, and 1 to account for the difference of the first day of the week.
3. Now that you've made this adjustment, you need to make sure you don't accidentally get a value which can't exist. If the adjusted day is less than zero, you add the count of `DayOfTheWeek` values to get it to wrap back around to a value which does exist.
4. Finally, you use the version of `first` which takes a predicate lambda to look at the list of all days, and return the first one where the ordinal matches the adjusted day.

Now, it's time to use your new function. At the bottom of the `main()` function, add the following:

```
val today = DayOfTheWeek.today()
val isWeekend =
    "It is${if (today.isWeekend) "" else " not"} the weekend"
println("It is $today. $isWeekend.")
```

Run again, and at the bottom of your output, you should see the following — or whatever values are appropriate values for the day of the week it is for you:

```
It is Monday. It is not the weekend.
```

Note: When you include an enum case directly in string interpolation, like you did here with `$today`, the name property of the enum case is automatically used when the string is created. If there's another property you'd prefer to have added to the string other than the name, you'll have to specify that with something like `${today.ordinal}`.

You can also add functions directly to the enum class which depend on a particular instance.

In `DayOfTheWeek`, below the end of the list of cases but above where you've added companion object, add a new function to help calculate how many days, including today, it is from the current instance until a given day of the week:

```
fun daysUntil(other: DayOfTheWeek): Int {  
    // 1  
    return if (this.ordinal < other.ordinal) {  
        // 2  
        other.ordinal - this.ordinal  
    } else {  
        //3  
        other.ordinal - this.ordinal + DayOfTheWeek.values().count()  
    }  
}
```

What's happening in this code?

1. First, you're checking whether the ordinal value of the current instance is less than the ordinal of the passed-in index.
2. In the case where the current ordinal is less than the passed-in ordinal, you simply subtract the current ordinal from the passed-in ordinal to get the number of days between them.
3. If the current ordinal is greater than or equal to the passed-in ordinal, you perform the same subtraction, but add the count of the days of the week so that the number is not zero or negative.

Now, add the following lines to the `main()` function, between the declaration of `isWeekend` and your last `println()` statement, to calculate how long it is until Friday:

```
val secondDay = DayOfTheWeek.Friday  
val daysUntil = today.daysUntil(secondDay)
```

Next, update the `println()` statement so it also prints out the new information you've given it:

```
println("It is $today. $isWeekend. There are $daysUntil days  
until $secondDay.")
```

Run `main()` again, and it will output something similar to this, based on what day it is in your world:

```
It is Monday. It is not the weekend. There are 4 days until Friday.
```

Note that if today is Friday, you'll see seven days instead of zero days until Friday, since you wrapped the count around if it was equal to today's day.

Using when with enum classes

One of the most powerful features of enum classes is how they combine with the `when` expression. You've already seen how this can be used on basic types like `Int` and `String`.

But having a bit of context as to what the options are in the `when` expression makes it far easier to read and reason about. What does this look like in practice?

Add a `when` expression to the bottom of the `main()` function which prints out some classic '90s song lyrics based on the value of `today`:

```
when (today) {  
    DayOfTheWeek.Monday -> println("I don't care if $today's  
blue")  
    DayOfTheWeek.Tuesday -> println("$today's gray")  
    DayOfTheWeek.Wednesday -> println("And $today, too")  
    DayOfTheWeek.Thursday -> println("$today, I don't care 'bout  
you")  
    DayOfTheWeek.Friday -> println("It's $today, I'm in love")  
    DayOfTheWeek.Saturday -> println("$today, Wait...")  
    DayOfTheWeek.Sunday -> println("$today always comes too late")  
}
```

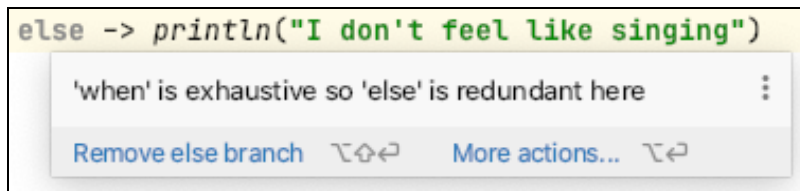
Run the `main()` function, and the appropriate line of the song will print at the end of your log based on the current day:

```
I don't care if Monday's blue
```

But what if you only want to print the lyric in certain circumstances? Well normally, you'd just add an `else` case. Try to do so at the end of your `where` expression:

```
else -> println("I don't feel like singing")
```

You'll get a warning from the compiler about this:



'when' is exhaustive so 'else' is redundant here

This is because you've already defined behavior for every case which exists. Delete or comment out all the cases except for Friday and the else case. The warning from the compiler will now go away, since the else case can cover at least one case.

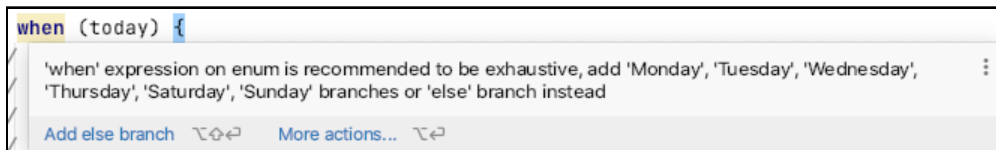
Run the `main()` function. If it's not Friday, you should see:

I don't feel like singing

If it is Friday, it'll print out:

It's Friday, I'm in love

You'll also get a warning if there are unhandled cases. Delete the else case, and you'll see the when get highlighted by the compiler. Hover over that highlight, and you'll see this warning:



'when' expression on enum is recommended to be exhaustive, add 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Saturday', 'Sunday' branches or 'else' branch instead

The code will still run, though! If it is Friday, you'll still see:

It's Friday, I'm in love

However, if it's not Friday, nothing will print. You'll still have the warning, but if you're not paying attention to it, you can easily miss it. Keep an eye out for this warning especially when adding new cases to an enum class, and make sure you've added appropriate handling for the new cases to your existing `when` expressions. What if you need to have something which defines more behavior than you can do easily in a single type, but still want to take advantage of the functionality that enum classes give you? A great way to do that is to use **sealed classes**.

Sealed classes vs. enum classes

As you saw briefly in the previous chapter, a **sealed class** has a limited number of direct subclasses, all defined in the same file as the sealed class itself. It's known as sealed as opposed to `final`, since although some subclassing is permitted (and in fact, required, as you'll see in a moment), the subclassing is extremely limited in scope.

The hope is that this technique allows programmers to take advantage of some of the flexibility of subclassing without permitting them to create massive inheritance trees which lead to terrible, incomprehensible code.

There are a few key points to know about sealed classes:

- They are **abstract**. This means that you can't instantiate an instance of the sealed class directly, only one of the declared subclasses.
- Related to that requirement, sealed classes can have `abstract` members, which must be implemented by all subclasses of the sealed class.
- Unlike enum classes, where each case is a single instance of the class, you can have multiple instances of a subclass of a sealed class.
- You can't make direct subclasses of a sealed class outside of the file where it's declared, and the constructors of sealed classes are always private.
- You *can* create indirect subclasses (such as inheriting from one of the subclasses of your sealed class) outside the file where they're declared, but because of the restrictions above, this usually doesn't end up working very well.

Creating a sealed class

Imagine you're working for a company that mostly works in U.S. dollars, but also accepts payments in Euros and some form of cryptocurrency.

Above the `main()` function, and below the end of your `DayOfTheWeek` enum class, add a new sealed class representing these accepted currencies:

```
sealed class AcceptedCurrency {  
    class Dollar: AcceptedCurrency()  
    class Euro: AcceptedCurrency()  
    class Crypto: AcceptedCurrency()  
}
```

In the `main()` function, add the following lines to the bottom:

```
val currency = AcceptedCurrency.Crypto()
println("You've got some $currency!")
```

Run the `main()` function, and you'll now see something like the following print out at the bottom:

```
You've got some AcceptedCurrency$Crypto@76ed5528!
```

Switching from an enum class to a sealed class means you lose all the nice convenience functions for things like name and order. You can see that as the name prints out as a bunch of gibberish.

Fortunately, sealed classes can have non-abstract properties with custom getters, and can also take advantage of when expressions.

Below the spot where `Crypto` is declared in the `AcceptedCurrency` sealed class, add the following property with a custom getter:

```
val name: String
get() = when (this) {
    is Euro -> "Euro"
    is Dollar -> "Dollars"
    is Crypto -> "NerdCoin"
}
```

Update the `println()` statement at the bottom of the `main()` function to take advantage of this new property:

```
println("You've got some ${currency.name}!")
```

Run the `main()` function again, and you'll see something a little more readable:

```
You've got some NerdCoin!
```

Since your company is U.S.-based, they'll want to know how much each of these currencies is worth in USD.

You can define a requirement for this by adding an abstract property on the sealed class, then overriding it in each of the subclasses.

Update the `AcceptedCurrency` sealed class to add an abstract `val`, and then override it in the three declared subclasses:

```
sealed class AcceptedCurrency {  
    abstract val valueInDollars: Float  
    class Dollar: AcceptedCurrency() {  
        override val valueInDollars = 1.0f  
    }  
    class Euro: AcceptedCurrency() {  
        override val valueInDollars = 1.25f  
    }  
    class Crypto: AcceptedCurrency() {  
        override val valueInDollars = 2534.92f  
    }  
    // leave the existing name property alone  
}
```

It would probably also help to know how much of a currency is being passed around with a single instance. You *can* add non-abstract `vals` and `vars` to a sealed class, as long as you provide them with an initial value.

Right below your abstract declaration of `valueInDollars`, add a new variable:

```
var amount: Float = 0.0f
```

Now that you have a place to store the value of a particular currency, you can calculate the total value of the accepted currency. You'll do that by adding a non-abstract function to your sealed class. Since every `AcceptedCurrency` subclass must provide a `valueInDollars` property, and all subclasses have access to the `amount` property you just added, you can use those at the `AcceptedCurrency` level to provide the same functionality across all classes.

Below the `name` property, add a new function to calculate the total value in dollars of a given currency:

```
fun totalValueInDollars(): Float {  
    return amount * valueInDollars  
}
```

Now, go back down to the `main()` function and add the following two lines to set an amount on the currency and print out the total value in dollars:

```
currency.amount = .27541f  
println("${currency.amount} of ${currency.name} is "  
    + "${currency.totalValueInDollars()} in Dollars")
```

Run the `main()` function, and at the bottom you should see:

```
0.27541 of NerdCoin is 698.1423 in Dollars
```

You're able to have as many instances as you want of any of the various subclasses of `AcceptedCurrency`, and those instances can store properties where enum class instances can't. Now that you know about sealed classes and some of the benefits and drawbacks of using them, you'll go back to dealing with enum classes for the remainder of this chapter. You'll start by looking at another important use of enum classes: **State machines**.

Enumeration as state machine

A **state machine** is essentially an exclusive list of possible states for a given system. Using an enum can make it more clear to the caller what state the system is in at any point.

Open up the provided **Downloader.kt** file, and you'll see a really simple example of this at the top with the `DownloadState` enum.

```
enum class DownloadState {  
    Idle,  
    Starting,  
    InProgress,  
    Error,  
    Success  
}
```

This has five exclusive states:

- `Idle`: Nothing has happened yet.
- `Starting`: The download is being started.
- `InProgress`: Data is actively being downloaded.
- `Error`: An error has occurred and caused the download to terminate.
- `Success`: The data download has completed successfully.

You'll also see a `Downloader` class. This has been provided to give an example of how enums can be used to glean information on the state of a system. This class does the hard work of figuring out how to adjust the state machine based on what's happening under the hood.

For now, the only thing you need to care about is the state your download is in. Fortunately, that's returned to callers of the main method on this class as part of a block indicating progress.

Go back to **main.kt**, and at the bottom of the `main()` method, add some new code:

```
Downloader().downloadData("foo.com/bar",
    progress = { downloadState ->
        //TODO
    },
    completion = { error, list ->
        // TODO
    })
```

This code pretends to download some data from the given URL, gives an update about the current state as it goes, and then tells you when the process is done either by displaying an error or a list of items.

First, replace the `TODO` in the completion handler with some code that handles completion:

```
error?.let { println("Got error: ${error.message}") }
list?.let { println("Got list with ${list.size} items") }
```

Next, you'll peer into the workings of the state machine in the progress handler. Replace the `TODO` there with a `when` statement, printing information about what's going on:

```
when (downloadState) {
    DownloadState.Idle -> println("Download has not yet started.")
    DownloadState.Starting -> println("Starting download...")
    DownloadState.InProgress -> println("Downloading data...")
    DownloadState.Error -> println("An error occurred. Download
terminated.")
    DownloadState.Success -> println("Download completed
successfully.")
}
```

Run your `main()` function, and at the bottom, you'll see output as the download progresses through each of the states:

```
"Downloading" from URL: foo.com/bar
Download has not yet started.
Starting download...
Starting download...
Downloading data...
[etc...]
Downloading data...
```



```
Got list with 100 items
Download completed successfully.
```

The Download class is designed to randomly throw an error about 10% of the time. When that happens, you can validate that your error handling code is working, since you'll see something like the following:

```
"Downloading" from URL: foo.com/bar
Download has not yet started.
Starting download...
Starting download...
Downloading data...
Downloading data...
[etc...]
Got error: Your download was eaten by a shark.
An error occurred. Download terminated.
```

Now, with the power of a simple when expression, you can easily handle all the various states which your download could be in.

Nullables and enums

Enums can also be dealt with at both the when level and as part of an API with nullability. In the Downloader class, instead of having an Idle option in DownloadState, you could express that nothing was happening by allowing the download state to be optional.

Give it a shot! In **Downloader.kt**, delete the Idle state from the DownloadState enum class.

Next, in the Downloader class, update the downloadState var to be optional, and null by default:

```
var downloadState: DownloadState? = null
```

Next, within the Downloader class, update the method signatures for downloadData and postProgress to use an optional DownloadState instead of a required one:

```
fun downloadData(
    fromUrl: String,
    progress: (state: DownloadState?) -> Unit,
    completion: (error: Error?, data: List<Int>?) -> Unit
) {
    // rest of method unchanged
}
```

```
...
private fun postProgress(
    progress: (state: DownloadState?) -> Unit
) {
    // rest of method unchanged
}
```

Now go back to **main.kt**. In the `main()` function, you'll now see an error in the `when` expression for downloading data:



Delete the line for handling the `Idle` state you removed, and replace it with handling for `null`:

```
when (downloadState) {
    null -> println("No download state yet")
    /// rest of when unchanged
}
```

Run the `main()` function one last time, and now, that same `when` expression is handling both `null` and non-`null` values coming into it:

```
"Downloading" from URL: foo.com/bar
No download state yet
Starting download...
```

Taking advantage of nullability with enum classes lets you represent state where you haven't received information, or have received unexpected information without having to explicitly create an "Idle" or "Unknown" state.

Challenges

1. Add a companion function to `DayOfTheWeek` which returns a nullable `DayOfTheWeek` based on a passed-in index. Do the same for a passed-in string.
2. Add a function to `DayOfTheWeek` to calculate how many days until the next weekend begins. Then, update your code so that the weekend is Wednesday and Thursday instead of Saturday and Sunday. Does it still work?

3. Create a way to add together the value of two `AcceptedCurrency` objects. Think about the following scenarios:
 - What should happen if both currencies are the same type?
 - What should happen if the currencies are of different types?
4. Create a function that can take a `List` of `AcceptedCurrency` objects and the cost of an item in Dollars, and return whether the user has sufficient funds in the list of currency objects to pay for what they're trying to buy.

Key points

- **Enum classes** are a powerful tool for handling situations where a piece of data will (or at least should) be one of a defined set of pre-existing values. Enum classes come with a number of tools for free, such as getting a list of all the declared cases, and the ability to access the order and names of the cases.
- **Sealed classes** are a powerful tool for handling situations where a piece of data will (or at least should) be one of a defined set of pre existing types.
- Both enum classes and sealed classes let you take advantage of Kotlin's powerful when expression to clearly outline how you want to handle various situations.
- Enum classes are particularly useful for creating, updating, and cleaning information about the current state in a **state machine**.

Where to go from here?

There are a few more places where you can learn more about enum classes and sealed classes:

- The official Kotlin documentation for **enum classes**: <https://kotlinlang.org/docs/reference/enum-classes.html>
- The official Kotlin documentation for **sealed classes**: <https://kotlinlang.org/docs/reference/sealed-classes.html>

You've spent the last few chapters learning about defining custom types with objects, classes and variants such as data classes, enum classes, and sealed classes. In the next chapter, you'll learn about defining custom types that focus on behavior using **interfaces**.